

1.0 Introduction

1.1 Project Overview

1.2 Background of the problem

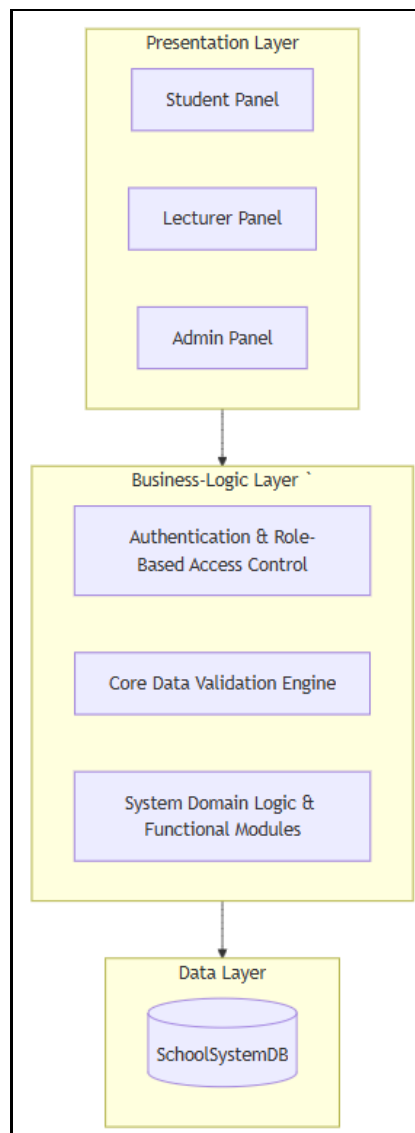
1.3 Research Questions

1.4 Project Objectives

2.0 System Architecture & Quality Evaluation

2.1 Proposed System Architecture

The proposed Student Information Management System (SIMS) is designed specifically for use in universities and colleges. As such academic institutions require this system to support essential operations such as student course enrolment and lecturer module uploads. SIMS accommodates three user roles which are Student, Lecturer and Administrator. The system is built upon a three-tier architecture comprising the Presentation Layer, Business-Logic Layer and Data Layer as illustrated in the diagram below.



Diagram

Presentation Layer

The Presentation Layer is responsible for managing all user interactions within the system. It consists of the ASPX pages designed for Students, Lecturers and Administrators. It provides the graphical interface through which users access system functionality. This layer captures user inputs through form fields, buttons and other interface components, while presenting information such as announcements, course lists and enrolment records. Its primary role is limited to handling user interface elements and forwarding requests to the Business Layer for processing.

Business Layer

The Business Layer is implemented within the code-behind files (.aspx.cs) and serves as the core processing component of the system. It is responsible for enforcing business rules, validating user requests and coordinating interactions between the Presentation Layer and the Data Layer. Before any operation is performed, this layer verifies that relevant conditions are satisfied. For example, enrolment requests are validated by checking the status of a course and ensuring that students cannot enrol in inactive courses. This layer ensures that system data remains accurate, consistent and compliant with institutional requirements by applying business rules and decision-making logic before database operations.

Data Layer

The Data Layer consists of the SQL Server database (**SchoolSystemDB**) which is responsible for the persistent storage and retrieval of system data. This layer stores information related to courses, enrolments, announcements and other entities required by the application. Database operations are performed using components such as SqlConnection and SqlCommand to enable data to be inserted, updated, retrieved and deleted as required. This layer is responsible for executing raw data access operations and maintaining the integrity of the underlying database.

2.2 Architectural Design Decisions

Three-tier architecture

The system follows a three-tier architecture, which is the Presentation Layer, Business-Logic Layer and Data Layer. Each layer has a single responsibility. Changes to business rules, such as restricting enrolment to active sessions or applying session-based data scoping contained within the Business Layer, without requiring modifications to the user interface or database schema. This separation enables multiple team members to work on the system simultaneously. For instance, one person can develop the user interface while another adjusts a query or validation rule, which only causes minimal conflict. The architecture also improves maintainability and testability, as logic is not tangled directly within page markup.

Role-based access control

Role-based access control is implemented because the system serves three distinct user types, which is Students, Lecturers, and Administrators. Each with different permissions and views. Students enrol in courses; lecturers grade assignments and administrators manage users and course allocations. This ensures that users see only functionality relevant to their role, thereby reducing confusion and the risk of unauthorized actions. Centralizing role checks provides a consistent security boundary across all pages within each role. This design also simplifies future extensions as new features for one role, such as a new Lecturer report, can be added without affecting the Student or Admin panels.

2.3 Architecture Evaluation Against Quality Attributes

2.3.1 Maintainability

The system implements a three-tier architecture, thereby improving maintainability by reducing dependencies between components. If changes made in one layer, the other layers will not be affected. This simplifies debugging, testing and future enhancements. Shared Master Pages such as StudentCourseMaster, LecturerCourseMaster and AdminMaster allow updates to be made in a single location and applied consistently across related pages. Maintainability is further strengthened through database-level constraints, including foreign keys, unique constraints and cascading operations, which enforce data integrity and reduce duplicated validation logic. Together, these decisions make the system easier to update, extend and troubleshoot over time.

2.3.2 Security

User passwords are stored as hashed values to ensure that credentials are never kept in plain text within the database. Role-based access control restricts system functions according to user roles (Student, Lecturer, and Admin) with authentication required before accessing any protected resources. Session variables are validated on every page request to prevent unauthenticated users from accessing restricted pages through direct URL manipulation. In addition, parameterized SQL queries are used throughout the system to mitigate SQL injection attacks. User-generated content and error messages are also encoded using `HttpUtility.JavaScriptStringEncode()` before rendering, reducing the risk of cross-site scripting (XSS) attacks.

2.3.3 Reliability

System reliability is strengthened through database-level integrity constraints, including foreign keys, NOT NULL constraints and unique constraints prevent invalid, duplicate or incomplete data from being stored. Cascading deletes rules maintain referential integrity by automatically removing dependent records, thereby preventing orphaned data when related entities are deleted. In addition, exception handling using try-catch blocks for SQLException enables the system to gracefully manage database errors without disrupting normal operation. User-friendly SweetAlert notifications provide clear feedback while preventing the exposure of internal system details. Together, these mechanisms help ensure consistent system behaviour, data accuracy and stable operation even when unexpected errors occur.

2.3.4 Performance

System performance is improved by retrieving only the data required for each operation. Database queries use WHERE clauses on frequently searched fields to reduce the amount of data processed and returned. This helps maintain fast response times, particularly for pages that are accessed regularly. In addition, UpdatePanel controls support partial-page updates through AJAX, thereby allowing only relevant sections of a page to refresh instead of reloading the entire page. As a result, the system reduces network traffic, improves responsiveness and provides a smoother user experience.

2.3.5 Scalability

The three-tier architecture allows multiple web servers to be deployed behind a load balancer while sharing a single database. SQL Server able to support a large number of concurrent connections and can scale vertically or horizontally as the number of students, lecturers and courses grows. Session-based data scoping using academic_session keeps the active working set stable over time as historical session data remains isolated and queryable without slowing down current-session queries as total data volume accumulates across multiple academic terms. As a result, the system can host a large number of users without requiring structural changes to the application.

2.4 Summary

3.0 Software Project Management

3.1 System Development Methodology

3.1.1 Agile Scrum Methodology

The agile Scrum is an iterative approach for the development of software that divides the project into short timeboxes called sprints. Rather than creating the entire system in one upfront design and development, the work is broken into a prioritized backlog of tasks and a functional system increment is delivered at the end of each sprint. Short review meetings are held on a regular basis to review the project progress and plans for the next sprints are adapted from the new knowledge, requirements and problems identified.

3.1.2 Justification of using Agile Scrum Methodology

Dividing into Multiple Modules

The Student Information Management System contains several sections (Admin, Lecturer or Student) which have their own sub-features. Agile enables each module to be created, tested and optimized independently of the rest of the system without specifying the rest of the system in full before it is developed. As seen in the project's WBS, the work is organized into module-level groups separating admin, lecturer and student panel, which map naturally onto Agile epics that can be broken down further into sprint-sized tasks.

Challenges broken down into smaller sprints

Each module can be split into smaller, manageable chunks of work that should be delivered using sprints. For instance, S9-S10 for the lecturer profile and calendar. This makes large and complex features such as building three separate role-based panels with shared but distinct functionality achievable in incremental steps rather than as one overwhelming task. It also enables the team to work on database design and authentication before implementing advance function.

Review Progress Regularly

In a system that includes many interconnected components, it is crucial to conduct regular sprint reviews that enable the team to demonstrate implemented functionality through gathering feedback and validating that the system conforms to specified requirements. For instance, this project had experienced refinements during development. As a result, the Agile approach issues such as requirement inconsistencies, design weaknesses and implementation defects to be identified and addressed at an early stage rather than at the end of development.

Early handling of bugs and changes

Each sprint produces a measurable system increment, enabling bugs to be identified and resolved shortly after the related code is implemented, when the development context remains clear and corrective actions are less costly. During this project, several issues emerged as a result of earlier design decisions. For example, removing the `lecturer_id` attribute from the Course table would have affected functionalities across the Student, Lecturer, and Admin panels, requiring extensive tracing and modifications throughout the system. These cross-cutting problems can be identified and addressed incrementally by utilizing Agile instead of during a single late-stage test phase when it would be difficult to resolve.

3.2 Functional Requirements

The functional requirements define the specific behaviours and operations of SIMS to meet the needs of its users. These requirements are derived from the system objectives and are organised by user role and module to ensure comprehensive coverage of all system functionalities. Each requirement is distributed across the three users as shown below.

Requirement ID	User Role	Functional Requirement
FR-01	All Users	Log in securely
FR-02	Admin	View dashboard (overview, statistic & upcoming items)
FR-03	Admin	Manage announcements
FR-04	Admin	Manage calendar events
FR-05	Admin	Manage programmes
FR-06	Admin	Manage courses
FR-07	Admin	Manage enrolment
FR-08	Admin	Manage users
FR-09	Admin	Generate student performance reports
FR-10	Admin	Manage payments
FR-11	Admin	Manage academic sessions
FR-12	Admin	Manage course assignments for lecturers.
FR-13	Lecturer	Post course-specific announcements
FR-14	Lecturer	View list of assigned courses
FR-15	Lecturer	View student grade performance

FR-16	Lecturer	Create and manage course modules
FR-17	Lecturer	View submitted assignments before grading
FR-18	Lecturer	Grade student assignment submissions
FR-19	Lecturer	Record attendance
FR-20	Lecturer	View course home (overview & statistics)
FR-21	Lecturer	View enrolled students per course
FR-22	All Users	View announcements and notifications
FR-23	All Users	Manage own profile information
FR-24	Students	View attendance records
FR-25	Students	View course home (overview & statistics)
FR-26	Students	View classmates enrolled in a course
FR-27	Students	View academic results
FR-28	Students	View GPA and CGPA
FR-29	Students	View list of enrolled courses
FR-30	Students	Request course enrollment
FR-31	Students	View course modules and materials
FR-32	Students	View and submit course assignments
FR-33	Students	View assignment feedback/marks
FR-34	Students	View and make payments

3.2.1 All Users Requirements

Requirements FR-01, FR-22, and FR-23 apply to all users. FR-01 ensures secure authentication before granting system access. FR-22 enables users to view announcements and notifications posted within the system. FR-23 allows users to manage their own profile information, including personal details and password changes. These foundational requirements support system security and user convenience.

3.2.1 Admin Requirements

Administrators hold the highest privileges and are responsible for system configuration and oversight. FR-02 provides a dashboard with system statistics. FR-03 and FR-04 enable management of announcements and calendar events. FR-05 through FR-07 cover programme, course, and enrolment management. FR-08 allows user account management. FR-09 enables performance report generation. FR-10 handles payment management. FR-11 manages academic sessions. FR-12 assigns courses to lecturers.

3.2.2 Lecturer Requirements

Lecturers manage course content and assess student performance. FR-13 allows posting course-specific announcements. FR-14 displays assigned courses. FR-15 enables viewing student grade performance. FR-16 supports creating and managing course modules. FR-17 and FR-18 cover assignment viewing and grading. FR-19 enables attendance recording. FR-20 provides course overview and statistics. FR-21 displays enrolled students per course.

3.2.3 Student Requirements

Students access course content and track academic progress. FR-24 allows viewing attendance records. FR-25 displays course overviews. FR-26 shows classmates in a course. FR-27 and FR-28 enable viewing academic results and GPA/CGPA. FR-29 displays enrolled courses. FR-30 allows requesting enrolment. FR-31 enables viewing course materials. FR-32 supports assignment viewing and submission. FR-33 displays feedback and marks. FR-34 enables payment viewing and processing.

3.3 Product Backlog

The product backlog serves as the central repository for all development tasks required to implement the Student Information Management System (SIMS). Each backlog item is assigned a unique identifier and is linked to its corresponding functional requirement to ensure complete traceability throughout the development lifecycle.

Backlog ID	Task	Requirement ID	Priority	Assigned Member	Sprint
PBL-01	Establish ER schema and SQL database design	N/A (Pre-Dev)	High	Justin Goh Han Yang	Pre-Sprint
PBL-02	Implement SHA-256 secure user login and role redirection	FR-01	High	Justin Goh Han Yang	Sprint 1
PBL-03	Develop Admin dashboard UI, overview metrics and upcoming items statistics	FR-02	High	Justin Goh Han Yang	Sprint 2
PBL-04	Build Admin system profiles	FR-23	High	Justin Goh Han Yang	Sprint 2
PBL-05	Develop an Admin program management module	FR-05	High	Amelia Tai Hui Xin	Sprint 3
PBL-06	Build academic session controls	FR-11	High	Amelia Tai Hui Xin	Sprint 4
PBL-07	Develop Admin user management module	FR-08	High	Chloe Tan Kai En	Sprint 4
PBL-08	Integrate Admin payment tracking pipelines	FR-10	Medium	Justin Goh Han Yang	Sprint 5
PBL-09	Develop Admin course management interfaces	FR-06	High	Amelia Tai Hui Xin	Sprint 6

PBL-10	Build course-to-lecturer assignment pipelines	FR-12	High	Amelia Tai Hui Xin	Sprint 6
PBL-11	Develop student enrollment management	FR-07	Medium	Amelia Tai Hui Xin	Sprint 6
PBL-12	Generate cumulative student performance metrics and reports	FR-09	Low	Yeoh Hao Jie	Sprint 7
PBL-13	Build calendar event creator module	FR-04	Medium	Aqil Zanieal Yeong bin Zainuddin Yeong	Sprint 8
PBL-14	Build Admin announcement management module	FR-03	Low	Aqil Zanieal Yeong bin Zainuddin Yeong	Sprint 8
PBL-15	Build lecturer profile	FR-23	High	Justin Goh Han Yang	Sprint 9
PBL-16	Build Lecturer calendar interface	FR-23	High	Amelia Tai Hui Xin	Sprint 10
PBL-17	Develop Lecturer dashboard and course overview interfaces	FR-14 & FR-20	High	Yeoh Hao Jie	Sprint 11
PBL-18	Implement enrolled student directory viewing per course	FR-21	High	Chloe Tan Kai En	Sprint 12
PBL-19	Develop an attendance recording for course	FR-19	High	Justin Goh Han Yang	Sprint 13

PBL-20	Develop lecturer view for overall student grade metrics	FR-15	Low	Yeoh Hao Jie	Sprint 14
PBL-21	Build course material uploading structures	FR-16	Medium	Aqil Zanieal Yeong bin Zainuddin Yeong	Sprint 15
PBL-22	Create student assignment submission tracking lists	FR-17	Medium	Aqil Zanieal Yeong bin Zainuddin Yeong	Sprint 15
PBL-23	Design grading and numeric input fields for instructors	FR-18	Medium	Aqil Zanieal Yeong bin Zainuddin Yeong	Sprint 15
PBL-24	Implement course-specific bulletin boards for announcements	FR-13	Medium	Amelia Tai Hui Xin	Sprint 16
PBL-25	Develop Student landing dashboard UI & Design student profile setting interfaces	FR-25 & FR-23	High	Justin Goh Han Yang	Sprint 17
PBL-26	Build student course directory	FR-29	High	Yeoh Hao Jie	Sprint 18
PBL-27	Build historical attendance tracking display for students	FR-24	High	Chloe Tan Kai En	Sprint 19
PBL-28	Design cumulative academic results dashboards	FR-27	High	Yeoh Hao Jie	Sprint 20
PBL-29	Build automated GPA and CGPA mathematical calculation scripts	FR-28	High	Justin Goh Han Yang	Sprint 20

PBL-30	Build student calendar interface	FR-25	Medium	Amelia Tai Hui Xin	Sprint 21
PBL-31	Implement digital student course enrollment request forms	FR-30	Medium	Aqil Zanieal Yeong bin Zainuddin Yeong	Sprint 22
PBL-32	Integrate student fee viewing and payment	FR-34	Medium	Aqil Zanieal Yeong bin Zainuddin Yeong	Sprint 22
PBL-33	Design classmate directory listings for enrolled students	FR-26	High	Chloe Tan Kai En	Sprint 23
PBL-34	Create course module listing and file downloading capabilities	FR-31	Medium	Aqil Zanieal Yeong bin Zainuddin Yeong	Sprint 24
PBL-35	Build assignment submission portal and file attachment fields	FR-32	Medium	Aqil Zanieal Yeong bin Zainuddin Yeong	Sprint 24
PBL-36	Implement published assignment grades and instructor feedback display	FR-33	Medium	Yeoh Hao Jie	Sprint 25
PBL-37	Build global announcements and notification centre feed	FR-22	Low	Amelia Tai Hui Xin	Sprint 26
PBL-38	Execute formal unit, integration and user acceptance testing (UAT)	N/A (Post-Dev)	High	All members	Post Sprint

The product backlog comprises 38 development tasks that comprehensively cover all functional requirements identified in Section 3.2. The Requirement ID column provides traceability to the backlog item from the requirement, which helps ensure that no functionality is missed during implementation.

Tasks are categorized into High, Medium or Low according to the importance in relation to system operation. High-priority tasks are intended for features such as authentication, database design and basic user management capabilities. Medium priority tasks involve areas of the supporting features like payment processing and assignment workflows and Low priority tasks include reporting and notification features.

The backlog is split into 26 sprints and related tasks are grouped together to ensure development efficiency. For instance, PBL-21, PBL-22 and PBL-23 will be performed in Sprint 15 to address the entire assignment submission and grading process. Tasks are distributed among team members based on skill level in order to distribute the workload.

Pre-sprint activities set up the database foundation whereas post-sprint activities include formal testing procedures. This is a structured process which ensures systematic progress towards full implementation of SIMS with accountability and traceability throughout the development lifecycle.

3.4 Sprint Planning

3.4.1 Sprint 1: Project Setup and Login

Sprint 1 establishes the system foundation by implementing the role-based selection and secure login mechanism. Users are presented with Student, Lecturer or Administrator cards to access. The login process incorporates SHA-256 encryption to ensure secure authentication. After successful login, users will be redirected to their respective role-based dashboards. This sprint also finalises the remaining planning and database design tasks from the pre-sprint phase, ensuring the team is prepared for subsequent development.

3.4.2 Sprints 2-8: Administrative Subsystem Development

These sprints are not based on specific features but are the backbone for SIMS administration. The development involves the core management functions such as user administration, programme management, course management, academic session control, lecturer allocation, payment monitoring, reporting and communication tools. Admin is given with CRUD operations to control academic and institutional processes effectively. These sprints collectively establish the backend infrastructure required to support lecturer and student functionalities.

3.4.3 Sprints 9-16: Lecturer Subsystem Development

This phase focuses on developing the functionalities required by lecturers to effectively manage their teaching and learning responsibilities. The subsystem encompasses features such as course administration, attendance tracking, assessment handling, distribution of learning materials, announcements and personal scheduling. These features are designed to facilitate meaningful interaction between lecturers and students. Upon completion of this phase, lecturers will have access to a dedicated workspace that streamlines administrative tasks.

3.4.4 Sprints 17-26: End-User/Student Subsystem Development

The final development phase introduces the student-facing component of the Student Information Management System (SIMS). The functionalities developed in this phase provide students with access to academic information, course enrolment, attendance records, assessment results, learning materials, payment services, and system notifications. The primary objective is to establish a self-service learning environment in which students are empowered to manage their own academic activities independently. Upon completion of these sprints, students will be able to access all essential resources and services through a single integrated platform.

3.5 Work Breakdown Structure

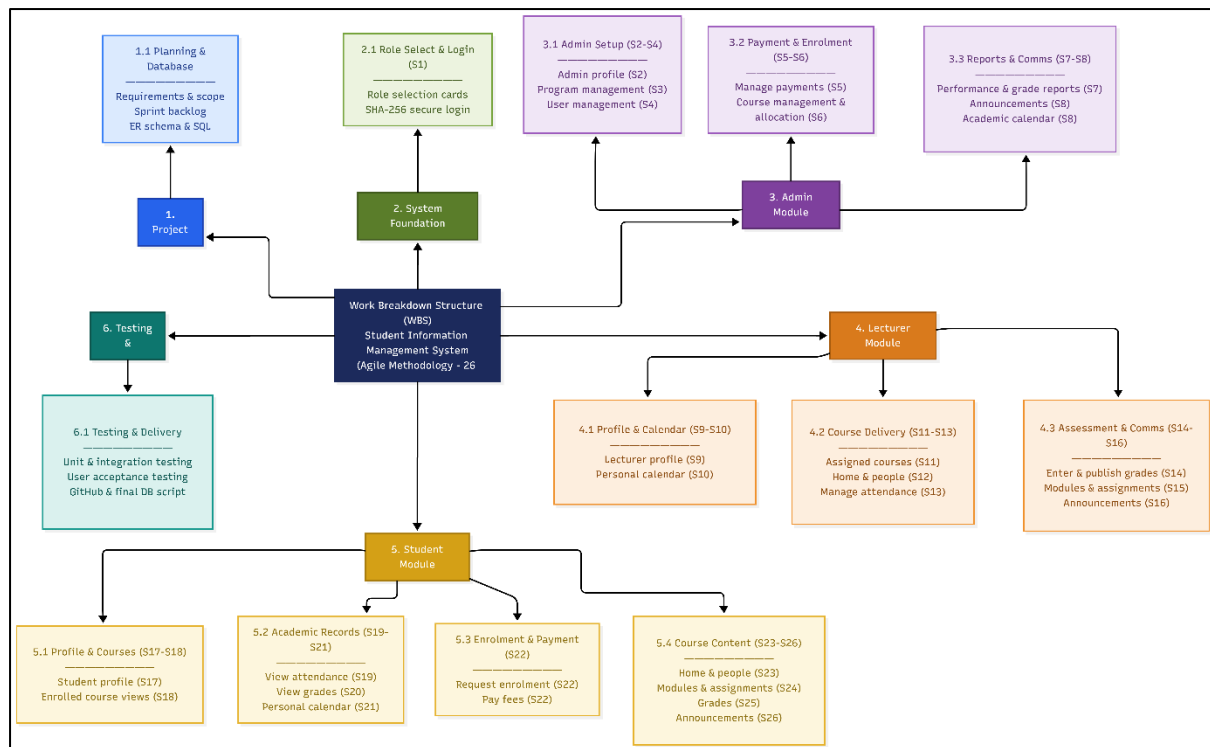


Diagram Work Breakdown Structure

The project was decomposed using an Agile sprint-based Work Breakdown Structure as illustrated in **Figure**. The WBS is organized into six sections with a total of **26 sprints** to ensure a balanced distribution between system foundation, functional development and quality assurance. For clarity, the system is organised into two main phases, which is System Foundation and Core Functional Modules.

Planning and database (1.0) focus on initial requirement gathering, sprint backlog definition and database design, including the ER diagram and SQL schema development. This ensures that the system architecture is properly defined before implementation begins.

System foundation (2.0) establishes essential technical infrastructure, including secure user authentication and role-based access control (RBAC). A SHA-256 hashing algorithm is implemented to ensure secure login functionality, thereby allowing users to access the system based on their assigned roles.

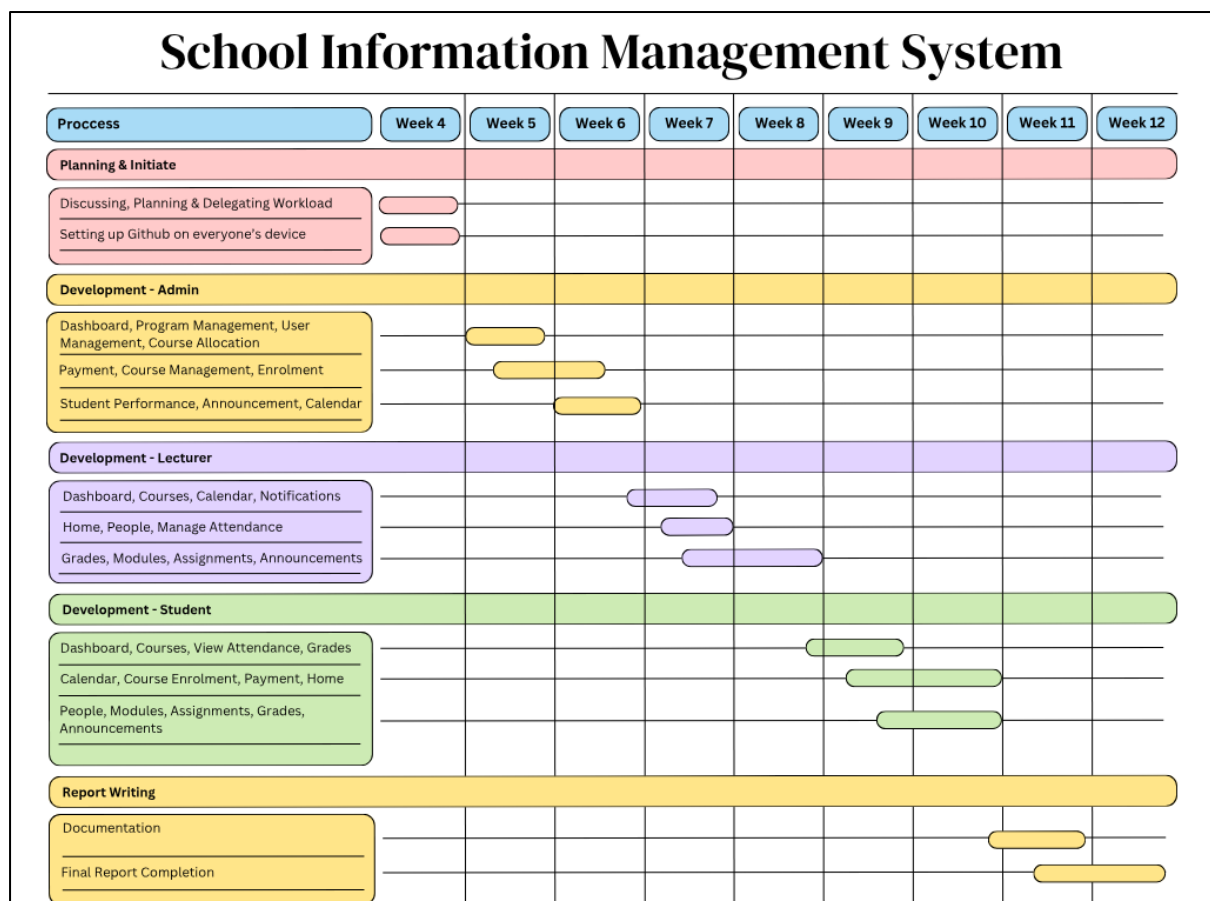
Admin module (3.0) is responsible for system administration and management functions, including Create, Read, Update and Delete (CRUD) operations for academic data and user management to support overall institutional control.

For the Lecturer module (4.0), lecturers are provided with an instructional workspace that enables lecturers to manage course content, assess student performance and handle grading activities efficiently.

The student module (5.0) represents the primary user-facing component of the system. The module allows students to access academic information, view results, view announcements and interact with essential learning-related services.

Testing and delivery (6.0) acts as the quality assurance layer of the system. It incorporates unit testing, integration testing and user acceptance testing (UAT) to ensure system reliability, correctness and readiness for deployment.

3.6 Gantt Chart



Diagram

Diagram shows the project timeline of developing SIMS in nine-week duration started from Week 4 to Week 12. The project utilizes a hybrid scheduling strategy where high-level phases progress sequentially, while internal sub-tasks leverage overlapping execution windows to optimize concurrency and resource allocation. The lifecycle is structured into five core phases as shown as below.

Phase 1: Project Initiation & Environment Configuration

The initial phase establishes the foundation for the project prior to system development to ensure clear accountability throughout the project. Concurrently, a version control system is configured across all team devices to standardise the repository structure and commit protocols, minimising the risk of merge conflicts during later integration stages.

Phase 2: Administrative Subsystem Development

Development commences with the administrative module, as the core entities managed by the admin serve as the foundation upon which all subsequent modules depend. Foundational features are implemented first, followed by concurrent development of supporting functionalities including payment processing, course management and enrolment handling. The phase concludes with secondary administrative features such as student performance tracking, announcement broadcasting and calendar scheduling.

Phase 3: Lecturer Subsystem Development

Following the completion of the administrative subsystem, development transitions to the lecturer panel. Core features, including the lecturer dashboard, course overview, calendar and notifications are implemented. Subsequently, operational modules such as the home interface, people directory and attendance management. The phase concludes with the development of more complex functionalities, including grade management, learning modules, assignment portals and course-specific announcements.

Phase 4: End-User/Student Subsystem Development

As the data structures have been established, student interactions can be seamlessly bound to existing backend data models. Development begins with read-oriented features such as the student dashboard, course view, attendance monitoring and grade lookup. Continuing with course enrolment, fee payment and personal scheduling. The phase concludes with access to learning materials, assignment submission, grade tracking and announcement feeds.

Phase 5: Technical Documentation & Final Review

This phase focus from development and towards project closeout and quality assurance. Activities during this period are a compilation of technical documentation, architectural specifications and codebase commenting. The team documenting the final project report, compiling test logs and preparing the deliverables for academic submission.

3.7 Configuration Management

Throughout the development for the SIMS, configuration management policies were enforced through GitHub. The strategy includes file organization, version control protocols, backup strategies and systematic change tracking.

3.7.1 File Organisation and Modular Architecture

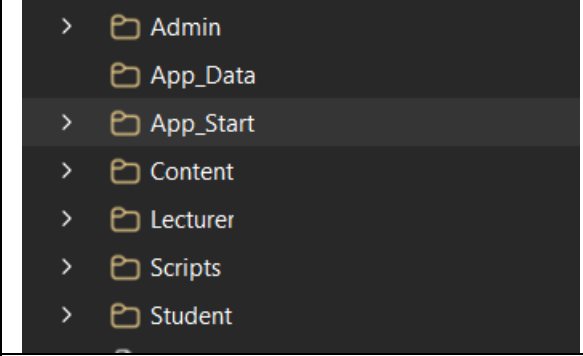
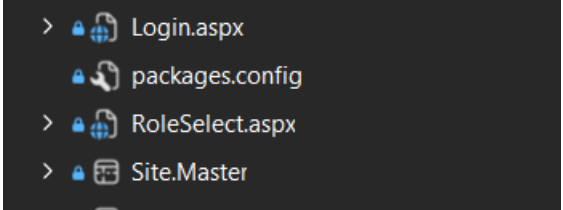
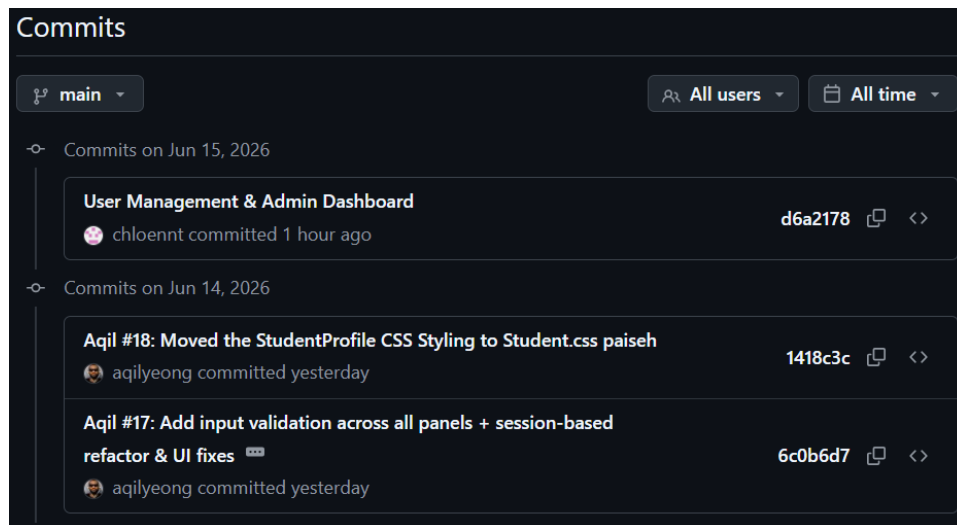
	
Figure	Figure

Figure and Figures shows the application is implemented using highly modular file organization pattern using an ASP.NET Web Forms framework.

The core source code is separated into different and independent subdirectories such as Admin, Lecturer and Student. This separation minimized the architectural blast radius. Multiple developers can work on different user modules simultaneously without risking structural overlap or directory conflicts through separating the codebase based on user roles.

Furthermore, global configurations and shared interfaces such as Login.aspx, RoleSelect.aspx and the shared layout template Site.Master are retained at the root level to provide unified gateways to these decoupled subdirectories.

3.7.2 Version Control Strategy



Diagram

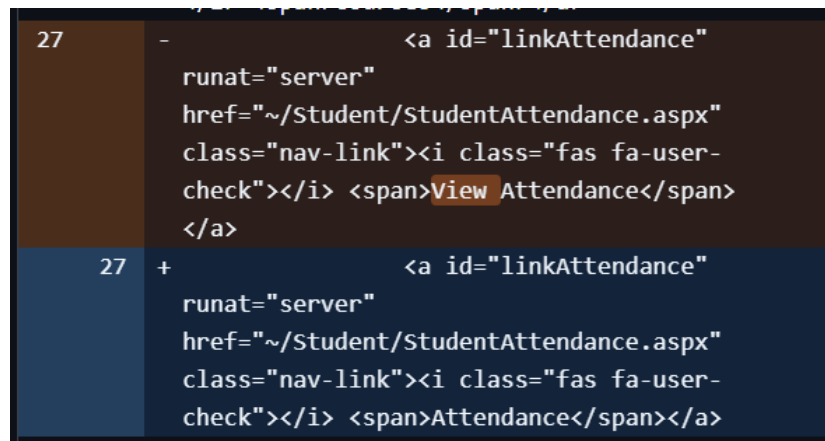
Git was utilized as the distributed version control system to track changes remotely through GitHub to manage code integration across the modular directories. Development was centralized within the Visual Studio environment. The local repository was linked to GitHub through integrated command-line interfaces to ensure maximum operational efficiency. The team members use manual commands such as `git pull` and `git push` in PowerShell terminals to fetch and upload the code without disrupting the development environment. This minimized context-switching and accelerated the deployment loop.

Every code update is pushed through the PowerShell terminal accompanied by a commit message. These messages explicitly documented the exact nature of the architectural changes made to the system. This approach ensures that any team member can trace the historical evolution of individual files or folders across the system lifecycle.

3.7.3 Backup and Recovery Approach

The configuration management plan ensures project resilience against data loss through a dual-layered backup topology. GitHub as the central repository ecosystem, `git push` serves as a cloud-based backup of the complete project state. If a developer experiences local hardware failure, the entire system state can be immediately recovered through a `git clone` operation. As Git tracks the snapshots of the code, the team is able to retain a chronological backup history. If a deployment introduces a critical bug into the Admin or Lecturer directories, the system can roll back to a known stable commit without disrupting unaffected directories.

3.7.4 Change Tracking and Auditing



```
27 - <a id="linkAttendance"
    runat="server"
    href="~/Student/StudentAttendance.aspx"
    class="nav-link"><i class="fas fa-user-check"></i> <span>View Attendance</span>
    </a>
27 + <a id="linkAttendance"
    runat="server"
    href="~/Student/StudentAttendance.aspx"
    class="nav-link"><i class="fas fa-user-check"></i> <span>Attendance</span></a>
```

Figure

Every modification to files or user directories requires a descriptive commit message documenting what changed and why. This creates an immutable audit trail within the repository history. It automatically generated a visual difference by highlighting every line of code added or removed as shown in Figure . Thereby, showing the changes of the code changes for peer review. If a configuration file is altered and breaks environmental dependencies, the team members can instantly cross-reference the change tracking logs to isolate the exact line of code, the responsible author and the timestamp of the modification as shown in Figure .

3.8 Collaboration Tools

A unified collaboration and version control tools was deployed to coordinate development tasks and maintain workflow synchronicity across the team. This infrastructure ensured that code integration, peer reviews and team communications were systematically tracked and managed throughout the project lifecycle.

3.8.1 Frontend Design Tool

Figma was utilized to design and prototype the user interfaces for SIMS. As a cloud-based design platform, it allowed multiple team members to work on the same design simultaneously, making collaboration more efficient. Team members could review layouts, suggest improvements and provide feedback directly within the design files. This reduced misunderstandings and ensured that everyone shared the same vision of the system before development began. The ability to view updates in real time also helped the team quickly refine designs and maintain consistency across different pages and user roles.

3.8.2 Code Review Tool

GitHub served as the primary platform for version control and collaborative code management throughout the SIMS development cycle. It allowed team members to work on different features independently using separate branches before merging their changes into the main codebase. This approach reduced conflicts and supported parallel development. GitHub also provided tools for reviewing code, tracking issues, and monitoring project progress. This ensures team members can easily identify modifications, resolve errors and collaborate more effectively to deliver a high-quality of the system.

3.8.2 Communication Tool

Discord served as the central communication platform for the SIMS project. Dedicated channels such as #resources, #frontend-design and #discussion were created to organise conversations according to specific topics. This structure helped prevent information overload and made important discussions easier to locate. Team members could share screenshots, files and error logs, which supported faster troubleshooting and knowledge sharing. For more complex technical discussions, Discord's voice call and screen-sharing features enabled team members to review problems together in real time. Important announcements, decisions and resource links were also pinned for easy reference.

3.8.3 Diagram Development Tool

Lucidchart was used to create and maintain system diagrams throughout the development of SIMS. The tool supported the creation of architecture diagrams, database models and workflow diagrams that helped team members visualise different aspects of the system. Its real-time collaboration feature allowed multiple team members to work on the same diagram simultaneously. These diagrams served as a shared reference throughout the project, thereby ensuring that all team members had a consistent understanding of the system structure, component relationships and overall design before implementation began.

3.9 Summary

4.0 Tools and Technology Selection

4.1 Technology Stack

The SIMS is developed utilizing multi-tier technology stack consisting of frontend technology, backend technology, database management system and supporting development tools. The selected technology was chosen due to the compatibility with the ASP.NET Web Forms framework, ease of development and suitability for managing academic data within an educational environment.

4.1.1 Frontend

The presentation layer handles the user interface components and client-side interactions throughout the Admin, Lecturer and Student panels. ASP.NET Web Forms (.aspx) is utilized as the primary server-side UI framework to generate dynamic web pages and handle user-driven interface events. Besides that, HTML (HyperText Markup Language) provides the foundational structural semantic markup for all web forms, input fields and dashboard containers. CSS (Cascading Style Sheets) enforces visual design, responsive layouts, grid systems and aesthetic consistency across the global application layout templates.

4.1.2 Backend

The business-logic layer implements the core business logic, user authentication pipelines and data processing operations. C# programming language serves as the primary object-oriented language used to program system logic, calculate metrics and handle operations. ASP.NET Code-Behind Files (.aspx.cs) are used to isolate backend server-side operational code directly from frontend presentation markup files to allow modular updates and cleaner debugging loops.

4.1.3 Database

The data storage layer utilises SQL Server and SQL Server Management Studio (SSMS). SQL Server is deployed as the core Relational Database Management System (RDBMS) responsible for executing relational queries, maintaining transaction integrity and enforcing data constraints. SSMS ensured database configuration, relational schema creation, optimization scripting and table maintenance.

4.1.4 Development and Collaboration Tools

A unified tooling is utilized to ensure a good operation. Visual Studio is configured as the Integrated Development Environment (IDE) for compiling ASP.NET architecture and executing manual Git integration commands. GitHub is hosted as the central cloud repository for distributing version control manage development function branch and conduct required code review. Discord as the team communication channel and lightweight agile task-distribution tracking system. Lucidchart as the modelling platform to map out chronological timelines and structural system architectures.

4.2 Justification of Selected Tools

4.2.1 Frontend and Backend Frameworks

The frontend layer is developed using ASP.NET Web Forms, HTML and CSS. ASP.NET Web Forms provides server-side controls that simplify the development of dynamic web pages while maintaining seamless integration with C# backend logic. HTML and CSS are both standard web technologies that are utilized to construct clean and modular layout components. These are integrated into global Master Pages (Site.Master) to enforce visual consistency, responsive design, and rapid user interface rendering across all module dashboards.

The backend is implemented using C# and ASP.NET code-behind files. This framework is suitable for enterprise-level applications. In this SIMS development project, the architecture ensures high maintainability and security across the three user-persona directories (/Admin, /Lecturer and /Student) by separating the user interface markup (.aspx) from the core server-side logic in C# code-behind files (.aspx.cs).

4.2.2 Database Management System

A relational database is critical for maintaining data integrity and executing complex data bindings such as processing fee payments, calculating GPA/CGPA scores and tracking attendance. SQL Server ensures robust data security and relational constraints, thereby making it suitable for managing student, lecturer, course, attendance and grading records. On the other hand, SSMS provides an optimal environment for writing database scripts, indexing data and managing the core Entity-Relationship (ER) schema.

4.2.3 Development and Collaboration Tools

Visual Studio serves as the primary integrated development environment (IDE) for coding, debugging and project management. GitHub is used for version control and source code

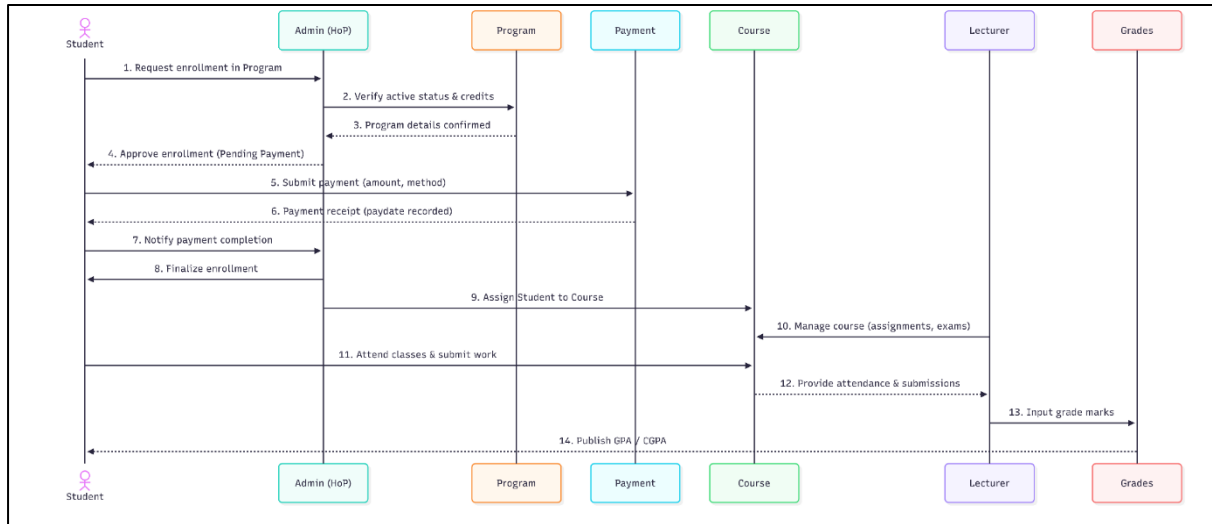
management which enables developers to track changes and maintain project history. Discord serves as a communication tool by setting up structured text channels to function as agile progress tracking boards. Additionally, diagramming tools such as Draw.io and mermaid are used to create system models including ER diagrams, class diagram, use case diagrams and system architecture diagrams.

4.3 System Design Models

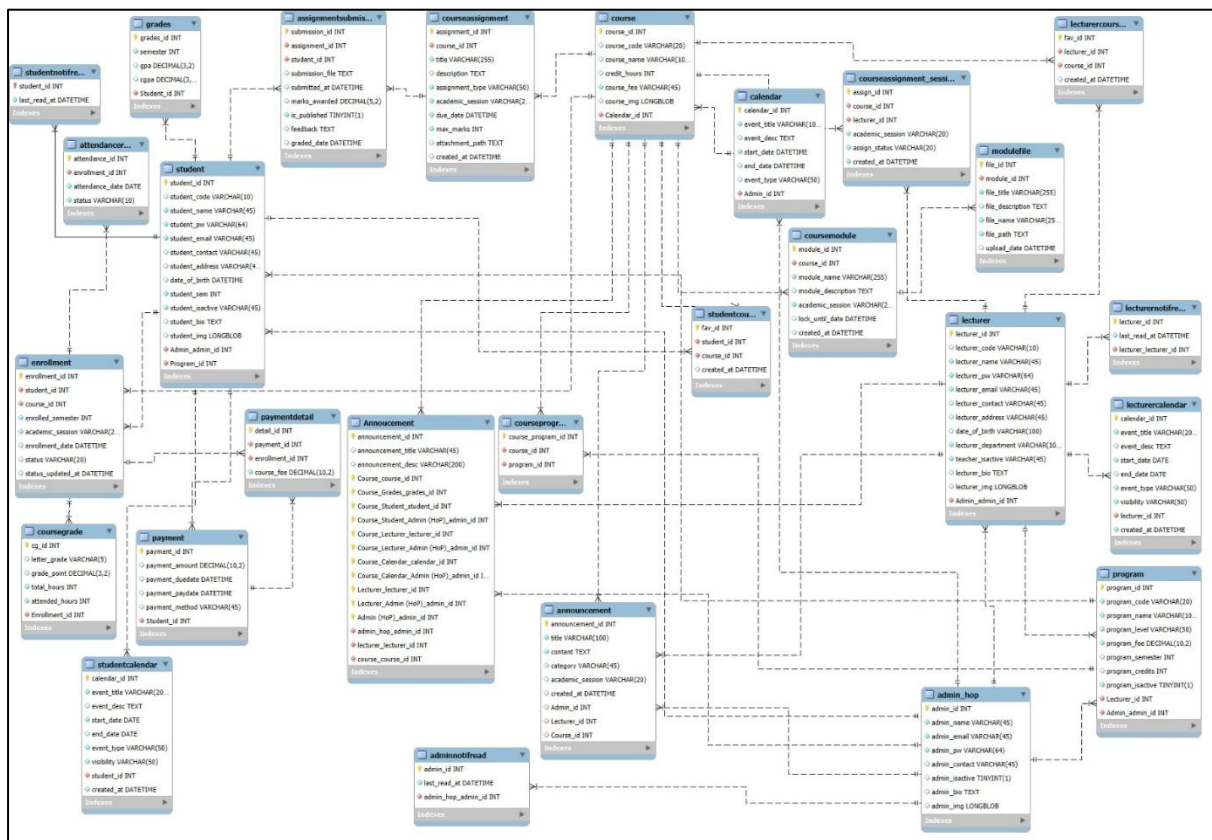
4.3.1 Use Case Diagram



4.3.3 Sequence Diagram

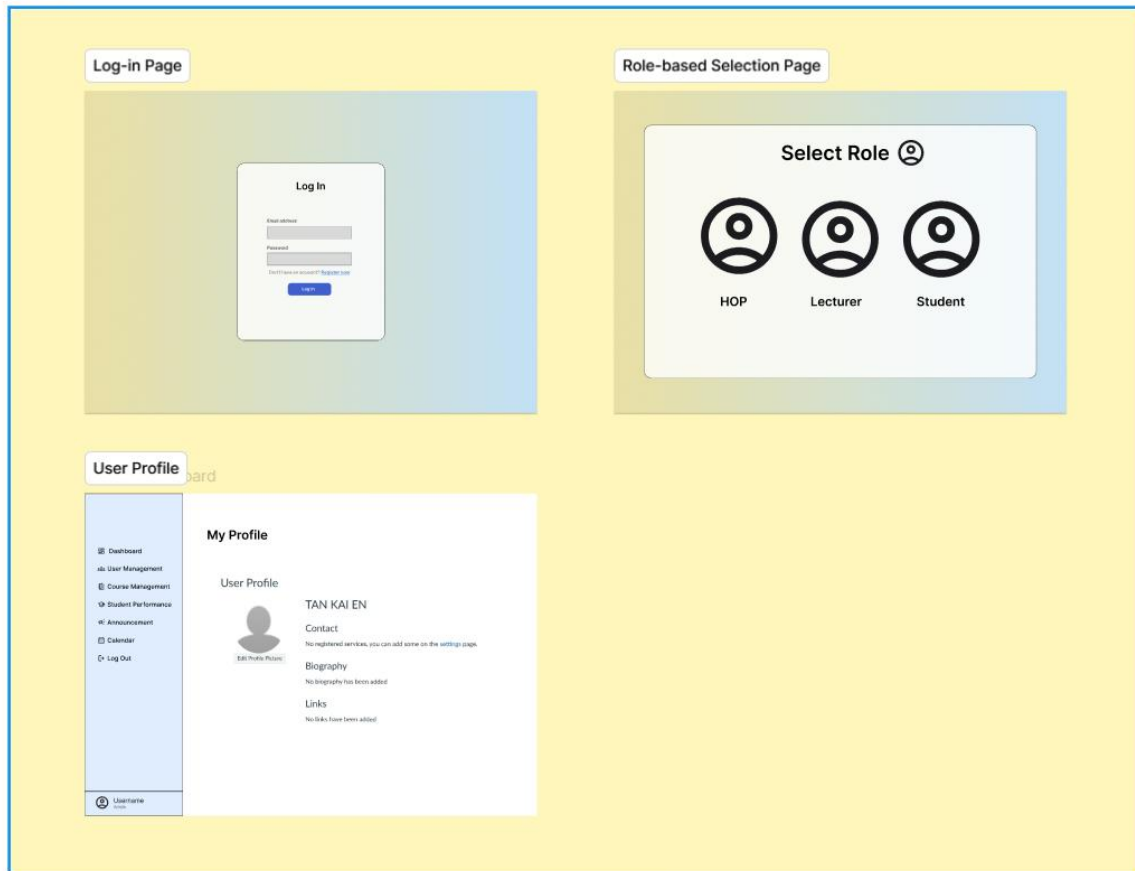


4.3.4 Database ER Diagram

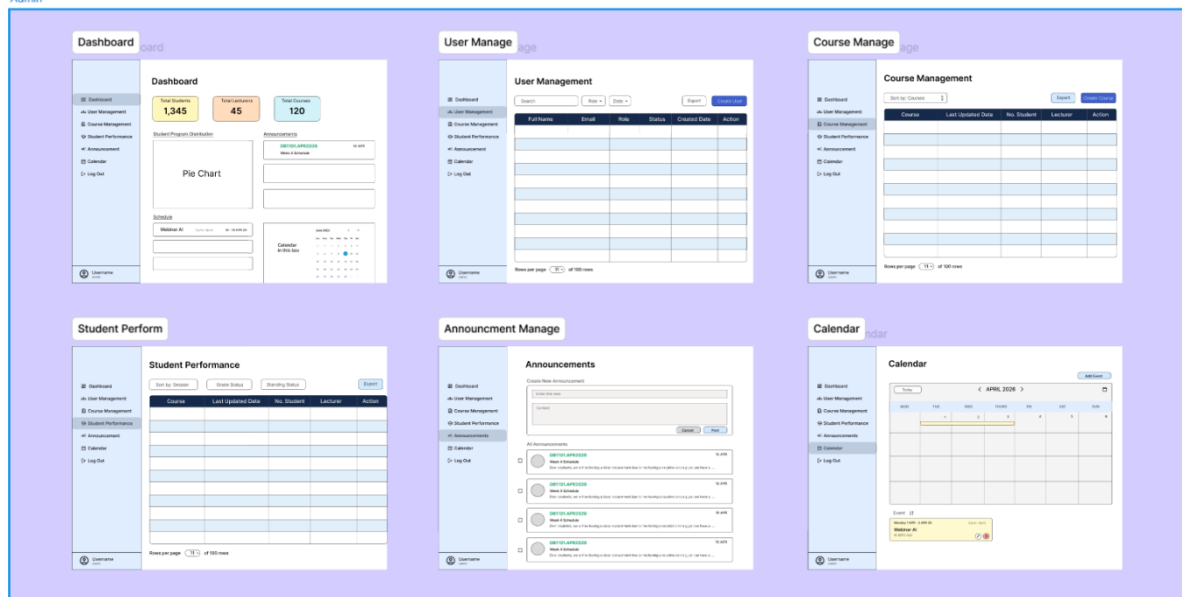


4.3.5 High-Fidelity Prototype

All Users



Admin



The Lecturer Dashboard includes the following modules:

- Dashboard:** Overview of published courses (5), students (250), and average attendance (94.8%). It also lists published courses and individual course modules.
- Calendar:** A calendar view for the current month (April 2020) showing course dates.
- Courses List:** A table listing all published courses with details like course code, name, level, and status.
- Individual Course Module:** A detailed view of a specific course module, including its description and associated resources.
- Course Module People:** A table showing the list of people enrolled in a specific course module.
- Attendance List:** A table showing attendance records for a specific course module, including student names and attendance percentages.
- Student Academic Perform:** A line graph showing the academic performance of students over time.

The Student Dashboard includes the following modules:

- Dashboard:** Overview of enrolled courses (5), average grade (3.84), and average attendance (88%). It also lists enrolled courses and individual course modules.
- Enrolled Courses List:** A table listing all enrolled courses with details like course code, name, level, and status.
- Attendance Overview:** A table showing attendance records for a specific course module, including student names and attendance percentages.
- Academic Result:** A table showing academic results for a specific course module, including student names and grades.
- Calendar:** A calendar view for the current month (April 2020) showing course dates.
- Enrolment Page:** A table showing enrollment details for a specific course module, including student names and enrollment status.

4.4 Summary

5.0 System Development

5.1 Module Implementation

5.1.1 Admin Module

5.1.2 Lecturer Module

5.1.3 Student Module

5.1.4 Additional Features

5.2 UI and UX Design Principles

Appendix

